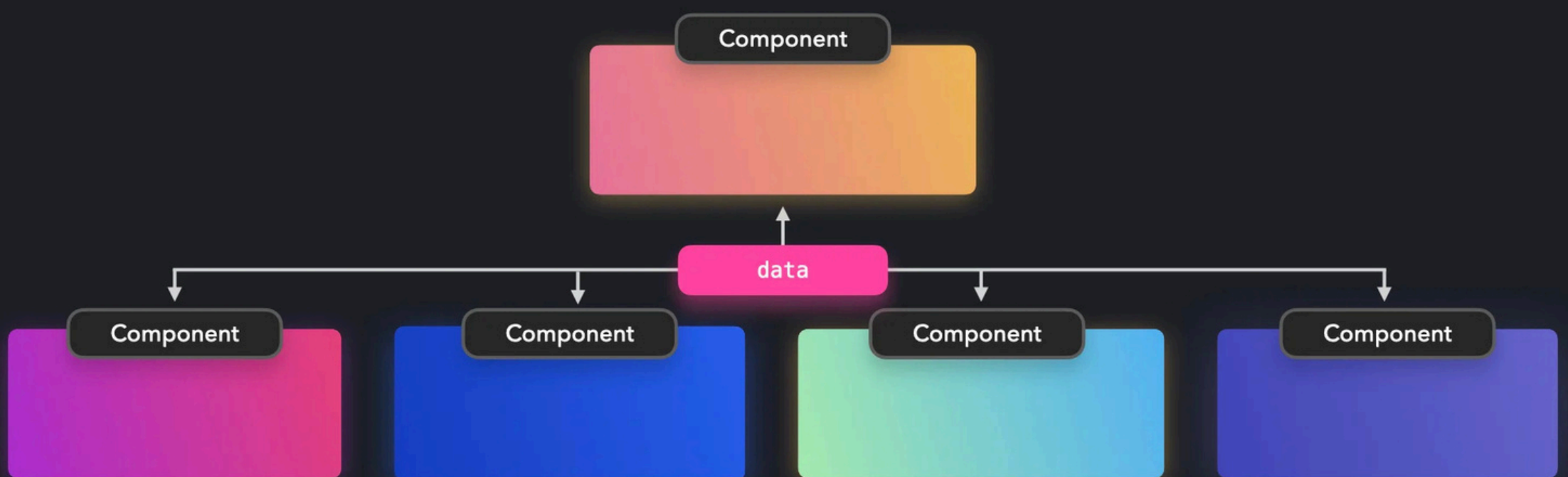


Day 30

Compound Components in React

A React Component Design Pattern



Introduction

React offers various **patterns** for building reusable and scalable components. One such powerful pattern is the **Compound Components Pattern**.

This pattern is particularly useful when you want to create highly **flexible** and **customizable** components.

The compound components pattern allows flexibility in how components are composed. The **parent** handles state and logic, while the **child components** focus on **presentation** and **interaction**. This makes it easy to manage complex UI structures in React.

If you're looking to create more complex UI structures while maintaining simplicity, this is an essential pattern to understand.

What Are Compound Components?

Compound components in React are components that work together to create a more flexible and cohesive UI.

The idea behind them is to **group multiple related components** that can communicate with each other, share state, and offer flexibility to the user on how to use them.

Think of compound components as a family of components that act as a **single unit**. These components allow for a more **declarative API** and make it easier to customize and configure complex behavior while maintaining simplicity.

In compound components:

- **Parent Component:** Acts as the container or the main component.
- **Child Components:** Are sub-components that share and use the parent component's state and logic.

Why Use Compound Components?

Compound components allow for:

- **Flexibility:** You can provide a highly customizable API where developers can structure components the way they want.
- **Separation of Concerns:** You split functionality across different components, making it easier to manage and maintain each.
- **Cleaner Code:** You avoid passing a lot of props down the component tree (prop drilling) by using compound components with shared state.
- **Reusable Logic:** You can reuse and combine the components to build various UI components, thus making them more scalable.

When to Use Compound Components?

Use compound components when:

- You need to create a reusable, customizable UI component with multiple related parts.
- You want to give the user flexibility in how they use or combine the components.
- You want to avoid prop drilling by having a parent component manage the shared state.

For example, a **Tabs** component is a classic use case. You can also use this pattern for forms, dropdown menus, accordions, modals, and more.

How Compound Components Work

Compound components typically work by:

- **Parent-Child Relationship:** The parent component manages the shared state, and the child components consume that state.
- **Shared State:** The parent manages the state, and children access this state through a context API, props, or by accessing the parent's functions directly.
- **Context API:** The React Context API is often used to share state and logic between the components without having to pass props manually to every child component.

Example: Building Compound Components

Let's build a **Tabs** component as an example of compound components. The component will have:

- A **Tabs** parent component to hold the state.
- **TabList**, **Tab**, and **TabPanel** as children components to display the tabs and their content.

Step 1: Creating the Parent Component (Tabs)

```
1 import React, { useState, createContext, useContext } from "react";
2
3 // Create a Context to manage state
4 const TabsContext = createContext();
5
6 const Tabs = ({ children }) => {
7   const [activeIndex, setActiveIndex] = useState(0);
8
9   // Function to change active tab
10  const selectTabIndex = (index) => setActiveIndex(index);
11
12  return (
13    <TabsContext.Provider value={{ activeIndex, selectTabIndex }}>
14      <div className="tabs">{children}</div>
15    </TabsContext.Provider>
16  );
17 };
18
19 export default Tabs;
```

In this parent **Tabs** component:

- **State Management:** The **Tabs** component manages the state for which tab is currently active. It uses the **useState** hook to keep track of the active tab with **activeIndex**, which initially starts at **0** (the first tab).
- **Context API:** The **TabsContext** is created using React's **createContext()** function. This context is used to pass down the **activeIndex** (the current active tab) and the **selectTabIndex** function (which allows changing the active tab) to the child components without needing to pass them manually as **props**.
- **Providing Context:** Inside the **Tabs** component, a **TabsContext.Provider** is used to wrap the child components (children). The value prop in **TabsContext.Provider** contains both the current active index and the function to change the active tab.

Step 2: Creating the TabList and Tab Components

Now, we will create the child components that will work inside **Tabs**.

```
1 const TabList = ({ children }) => {
2   return <div className="tab-list">{children}</div>;
3 };
4
5 const Tab = ({ index, children }) => {
6   const { activeIndex, selectTabIndex } = useContext(TabsContext);
7
8   const isActive = activeIndex === index;
9
10  return (
11    <button
12      className={`tab ${isActive ? "active" : ""}`}
13      onClick={() => selectTabIndex(index)}
14    >
15      {children}
16    </button>
17  );
18 };
19
20 export { TabList, Tab };
```

TabList Component: This is a simple container component that holds the **Tab** components. It accepts **children** as a prop, which can be one or more **Tab** components. It's styled as a list of tabs.

In the **Tab** Component:

- The **Tab** component uses `useContext(TabsContext)` to access the shared state (**activeIndex**) and the method to change the active tab (**selectTabIndex**).
- The **isActive** constant checks if the current Tab's index matches the **activeIndex**. If they match, it means this Tab is the active one.
- The **Tab** component renders a button. The **className** includes a conditional "**active**" class if the tab is currently selected. When the button is clicked, it triggers **selectTabIndex**, passing the current index, which changes the active tab.

Step 3: Creating the TabPanel Component

```
1 const TabPanel = ({ index, children }) => {  
2   const { activeIndex } = useContext(TabsContext);  
3  
4   return activeIndex === index ? <div className="tab-panel">{children}</div> :  
   null;  
5 };  
6  
7 export { TabPanel };
```

- Like the **Tab** component, the **TabPanel** also consumes the **TabsContext** using **useContext**. It needs to know the current active tab (**activeIndex**).
- The **TabPanel** checks if its index matches the **activeIndex**. If they match, it renders the panel's content ({children}). If not, it renders **null**, meaning nothing is displayed for that panel if it's not active.

Step 4: Using the Compound Components Together



```
1 import React from "react";
2 import Tabs, { TabList, Tab, TabPanel } from "../Tabs";
3
4 function App() {
5   return (
6     <Tabs>
7       <TabList>
8         <Tab index={0}>Tab 1</Tab>
9         <Tab index={1}>Tab 2</Tab>
10      </TabList>
11
12      <TabPanel index={0}>Content for Tab 1</TabPanel>
13      <TabPanel index={1}>Content for Tab 2</TabPanel>
14    </Tabs>
15  );
16 }
17
18 export default App;
```

- **Tabs Component:** Acts as the parent component and manages the **shared state** for the active tab. It provides the context for its children (**TabList**, **Tab**, and **TabPanel**).
- **TabList** and **Tab**: Inside **TabList**, two **Tab** components are defined with different index values (0 and 1). These are the **clickable tabs**. When clicked, they change the active tab.
- **TabPanel**: Corresponds to the content for each tab. When a tab is selected, the corresponding **TabPanel** displays its content. If **Tab 1 is active**, the content for Tab 1 ("Content for Tab 1") is shown.

A perfect example is the **Tab component from the Headless Ui library.**

Real-World Use Cases

- Accordion: Different sections expand and collapse.
- Dropdown: A menu that can expand or collapse, with different sections.
- Wizard Forms: A form that has multiple steps where each step corresponds to a different panel.
- Modals: A complex modal window that has different sections or tabs.

Best Practices for Compound Components

- **Use Context API:** This helps you avoid prop drilling and keeps the state within the component tree without unnecessary prop passing.
- **Keep Child Components Stateless:** The parent should manage all the state and logic. The children only consume the state and call the parent's methods.

Congratulations, We Made It!!!

I am more than excited for this journey, how we started and how we are going (Learning is continuous right).

If these 30 days has been helpful to you, kindly leave your thoughts in the comments, I would love to read them (for encouragement and improvement).

Once again, thank you for journeying with me! I love love love y'all 100



I hope you found this material
useful and helpful.

Remember to:

Like

Save for future reference

&

Share with your network, be
helpful to someone 🙌

Hi There!

Thank you for reading through

Did you enjoy this knowledge?

 Follow my LinkedIn page for more work-life balancing and Coding tips.

 LinkedIn: Oluwakemi Oluwadahunsi

kodemaven-portfolio.vercel.app